Security Vulnerabilities in Smart Contracts for Identity Management

Project Submitted To

ADITYA UNIVERSITY

In partial fulfilment of the requirement for the degree of

MASTER OF SCIENCE IN CYBER SECURITY & DIGITAL FORENSICS

Submitted by

[STUDENT NAME]

(Registration No.: [TO BE PROVIDED])

Under the Guidance of

[GUIDE / SUPERVISOR NAME]

Assistant Professor

DEPARTMENT OF FORENSIC SCIENCE

SCHOOL OF SCIENCES

ADITYA UNIVERSITY

[MONTH AND YEAR]

DEPARTMENT OF CYBER SECURITY & DIGITAL FORENSICS
ADITYA UNIVERSITY



CERTIFICATE

This is to certify that the project report entitled "Security Vulnerabilities in Smart Contracts for Identity Management" is submitted by [STUDENT NAME] (Registration No.: [TO BE PROVIDED]) in partial fulfilment of the requirements for the award of Master of Science in Cyber Security & Digital Forensics. This certificate page is an unsigned template placeholder and requires institutional review, verification and signatures before submission.

Signature
PROJECT GUIDE

Signature
Mr. VILAS ANIL CHAVAN
ASSOCIATE DEAN
SCHOOL OF SCIENCES
ADITYA UNIVERSITY

DEPARTMENT OF FORENSIC SCIENCE, SCHOOL OF SCIENCES, ADITYA UNIVERSITY



CERTIFICATE

This is to certify that the project report entitled "Security Vulnerabilities in Smart Contracts for Identity Management" is a record of work submitted by [STUDENT NAME] (Registration No.: [TO BE PROVIDED]) under the guidance of [GUIDE NAME]. The guide and examiner approval fields remain intentionally blank; this generated document does not assert that any certificate has been approved or signed.

Project Guide

Internal Examiner **External Examiner**

DECLARATION

I, [STUDENT NAME] (Registration No.: [TO BE PROVIDED]), declare that I have reviewed this report entitled “Security Vulnerabilities in Smart Contracts for Identity Management” and will confirm its originality, authorship and submission history before signing. This declaration is an unsigned placeholder and must be completed only after the student has verified that each statement is true.

Date:

Name:

Signature:

ACKNOWLEDGEMENT

Acknowledgement draft for completion by the student: I gratefully acknowledge [GUIDE / SUPERVISOR NAME] for academic guidance, and [OTHER CONTRIBUTORS, IF APPLICABLE] for their contribution. Please replace this draft with an accurate, personally verified acknowledgement; no individual contribution is asserted by this placeholder.

I also acknowledge the academic and technical resources used to implement and evaluate this local research prototype. Add or remove acknowledgements to reflect actual assistance received.

The student should review this page, identify all contributors accurately, and remove this instruction before final submission.

[STUDENT NAME]

TABLE OF CONTENTS

S.NO.	TITLE OF CONTENTS	PAGE NO.
1	ABSTRACT	xii
2	LIST OF ABBREVIATIONS	xii
3	1. INTRODUCTION	1
4	2. REVIEW OF LITERATURE	4
5	3. METHODOLOGY	6
6	4. OBSERVATIONS	10
7	5. RESULTS	13
8	6. CONCLUSION	16
9	7. DISCUSSION	18
10	8. REFERENCES	20
11	APPENDICES	21
12	APPENDICES	21

DETAILED AUTOMATIC CONTENTS

LIST OF ABBREVIATIONS xii

1. INTRODUCTION 1

1.1 Background 1
1.2 Research background and problem statement 1
1.3 Motivation and significance 2
1.4 Aim and objectives 2
1.5 Research questions 2
1.6 Scope and assumptions 3
1.7 Research significance and contribution 3
2. REVIEW OF LITERATURE 4
2.1 Decentralized identity and credential models 4
2.2 Smart-contract security research 4
2.3 Comparative literature analysis 4
2.4 Synthesis and research gap 5
3. METHODOLOGY 6
3.1 Research design 6
3.2 Research questions and evaluation criteria 6
3.3 Development environment and tools 6
3.4 System architecture and workflow 7
3.5 Contract pairing and implementation method 8
3.6 Experimental procedures 8
3.7 Data collection, metrics and analysis 9
3.8 Research ethics, safety and reproducibility 9
4. OBSERVATIONS 10
4.1 Identity contract behavior observed in source 10
4.2 Credential registry behavior observed in source 10
4.3 Frontend and wallet integration observed 11
4.4 Deployment and chain observations 11
4.5 Automated test coverage observed 12
5. RESULTS 13
5.1 Recorded test and experiment result summary 13
5.2 Access-control and input-validation observations 13
5.3 Gas observations 14
5.4 Static-analysis results 14

5.5 Figure and evidence interpretation 15
6. CONCLUSION 16
6.1 Research summary 16
6.2 Achievement of objectives and answers to research questions 16
6.3 Major findings and contribution 16
6.4 Limitations and future work 17
7. DISCUSSION 18
7.1 Interpretation in relation to prior work 18
7.2 Security improvement and gas trade-off 18
7.3 Privacy, data integrity and lifecycle considerations 18
7.4 Validity, reproducibility and generalization 19
7.5 Evidence gaps and submission readiness 19
8. REFERENCES 20
APPENDICES 21
Appendix A. Transaction receipt evidence from the stored local experiment report 21
Appendix B. Automated test inventory 21
Appendix C. Representative source excerpts 22
Appendix D. Reproduction and submission-completion notes 22

LIST OF TABLES

Table No	Table Caption	Page no
1	Selected literature and standards relevant to the implemented study	4
2	Verified implementation environment and responsibilities	6
3	Paired outcomes and receipt gas from the recorded experiment run	13
4	Recorded successful baseline gas use	14
5	Recorded Slither findings by contract	14
6	Local experiment transaction hashes and measured gas	21
7	Test cases present in the checked-in Hardhat suite	21

LIST OF FIGURES

Figure No	Figure Caption	Page no
1	Research and experimental evidence workflow	6
2	Implemented frontend, wallet, local chain and contract architecture	7
3	Tested identity modification/deletion authorization boundary	10
4	Credential issuance, verification, revocation and expiry controls	11
5	Gas used by paired transactions in the recorded run	14

LIST OF ABBREVIATIONS

ABI — Application Binary Interface

API — Application Programming Interface

DID — Decentralized Identifier

EVM — Ethereum Virtual Machine

JSON — JavaScript Object Notation

RPC — Remote Procedure Call

SSI — Self-Sovereign Identity

UI — User Interface

VC — Verifiable Credential

ABSTRACT

This study implemented and evaluated four Solidity smart contracts to examine identity and credential-management security on a local Ethereum-compatible blockchain. The paired contracts comprise deliberately vulnerable teaching implementations and access-controlled counterparts. Identity tests cover unauthorized modification and deletion, empty or zero-valued input, overlong labels, duplicate registration, and owner-authorized lifecycle operations. Credential tests cover issuer authorization, malformed and duplicate credentials, unauthorized revocation, and expiry. The experimental harness deploys fresh instances to Hardhat chain ID 31337, uses synthetic data and separate deterministic signers, and records transaction receipts, gas used, and before/after state in JSON and CSV reports. The recorded experiment artifact reports seven paired scenarios and four unique unauthorized-operation classes: all four unauthorized calls succeeded against the vulnerable contracts and were blocked by the secure variants. A recorded automated suite reported 20 passing tests and no failures. Single-run gas observations show that secure checks add modest gas to successful owner operations while reverted attack calls still consume gas. Slither output reports timestamp-related findings for credential expiry but no identity authorization warning; tests, rather than static analysis, establish the demonstrated access-control flaws. The prototype is not a production identity system: it stores synthetic labels and deterministic commitments on a public-readable ledger and does not implement the W3C DID or Verifiable Credentials protocols. Results are limited to deterministic local experiments; browser-wallet transactions were not manually verified as part of the evidence set.

Keywords: blockchain security; smart contracts; identity management; access control; decentralized identity; credential registry; Hardhat; Slither.

1. INTRODUCTION

1.1 Background

A blockchain is a replicated transaction ledger maintained by a network of participants. Cryptographic linking, consensus and distributed validation make accepted records tamper-evident under the network assumptions; they do not make every application deployed on the ledger correct or confidential [1]. Smart contracts are programs whose state transitions are executed by the blockchain virtual machine. Their public interfaces make authorization and input-validation decisions part of the application's security boundary.

Identity management comprises creation, use, update, verification and retirement of identity-linked records. Decentralized identifier standards describe a broader ecosystem of identifiers and controllers [2], while surveys of self-sovereign identity discuss additional components beyond an on-chain record [3]. Verifiable-credential standards describe issuer, holder and verifier roles [4]. The project implements a smaller teaching model: an Ethereum address is the subject key; an identity record holds a display label and a bytes32 commitment; a credential registry records a credential identifier, subject, issuer, expiry and revocation state. It is not a standards-conformant DID or verifiable credential implementation.

1.2 Research background and problem statement

On an open smart-contract interface, a caller can invoke any externally callable function unless the contract enforces a suitable authorization rule. An identity update or deletion function that accepts a target subject but fails to bind the caller to that subject can permit an unrelated account to change or remove another account's record. In a credential registry, an unrestricted issuance function can allow forgery, while unrestricted revocation can deny a subject the use of an otherwise valid credential. Missing validation can also admit malformed data, ambiguous records or overwritten identifiers.

The implemented research problem is whether explicit subject/issuer authorization and basic input/lifecycle validation change these observed outcomes when vulnerable and secure contracts are subjected to the same synthetic local test scenarios. The work does not test a public identity service, production key custody, consensus security, privacy-preserving proofs or a live Ethereum deployment.

1.3 Motivation and significance

Identity and credential records may influence access decisions even when the underlying application stores only references or commitments. Immutability can preserve an attack's effects as well as legitimate updates; a mapping declared private is not secret from chain observers, and emitted events and transaction input are public. A controlled vulnerable-versus-secure comparison makes caller checks visible, provides reproducible state-level evidence, and illustrates why static-analysis output should be interpreted alongside targeted tests [5]–[7].

1.4 Aim and objectives

The aim was to implement and empirically compare vulnerable and access-controlled identity and credential smart contracts in a reproducible local Hardhat environment. The objectives were:

- Implement paired vulnerable and secure Solidity contracts while preserving the deliberately insecure versions as isolated teaching artifacts.
- Test identity subject authorization for updates and deletion, and compare input and duplicate-registration behavior.
- Test credential issuer authorization, revocation authority, identifier/subject/expiry validation and expiry-boundary behavior.
- Collect transaction receipts, state changes, gas units and automated test outcomes using synthetic data.
- Run and interpret recorded Slither analyses, including detector limitations and timestamp observations.
- Provide a local Vite/Ethers/MetaMask interface for research workflows without claiming unverified manual wallet use.

1.5 Research questions

- RQ1: Do the implemented authorization checks prevent the tested non-owner identity updates and deletions?
- RQ2: Do issuer and revocation controls prevent the tested unauthorized credential issuance and revocation?
- RQ3: How do the paired contracts differ in accepting invalid identity and credential inputs and duplicate identifiers?
- RQ4: What gas-use differences were recorded for the tested operations, and what can—and cannot—be inferred from one local run?
- RQ5: How did the recorded Slither findings compare with the test-demonstrated access-control behavior?

1.6 Scope and assumptions

The scope is limited to four Solidity contracts compiled with Solidity 0.8.24, Hardhat 2.26.3, Ethers.js 6.13.4 and Vite 8.3.5. Tests use synthetic labels, hashes, addresses and deterministic local signers on chain ID 31337. The local RPC is configured as http://127.0.0.1:8545. The threat actor is an arbitrary transaction sender without the target subject's key or credential-registry administrator authority. Tests assess direct contract calls, not phishing, key theft, governance capture, client compromise, consensus attacks or cross-chain behavior.

1.7 Research significance and contribution

The project's contribution is an implemented, paired teaching testbed and a traceable dataset connecting function-level authorization rules to transaction outcomes and state changes. Its strongest conclusion is narrowly empirical: in the recorded local run, the selected unauthorized calls succeeded on the vulnerable contracts and reverted on the access-controlled counterparts. This is not a general security proof, an audit of an external protocol, or a statistical claim about all contracts.

2. REVIEW OF LITERATURE

2.1 Decentralized identity and credential models

Mühle et al. survey essential components of self-sovereign identity and organize a rapidly developing area around the roles and mechanisms needed for user-controlled identity [3]. Their work is useful for distinguishing a complete identity ecosystem from a single on-chain record. The present prototype does not implement identity recovery, DID resolution, wallet interoperability, selective disclosure or governance; it tests a small subset of record-lifecycle authorization.

The W3C DID Core Recommendation defines DID syntax, a data model, DID documents, operations and resolution, with the intention that a controller can prove control without dependence on a single centralized registry [2]. The W3C Verifiable Credentials Data Model 2.0 defines issuer, holder and verifier roles, credential claims and security/privacy considerations [4]. These standards are conceptual boundaries, not implementation claims: this project's credential registry stores a bytes32 ID and lifecycle metadata, not a standards-encoded signed credential or DID document.

2.2 Smart-contract security research

Atzei, Bartoletti and Cimoli provide a systematic survey and taxonomy of attacks on Ethereum smart contracts [5]. The broad lesson relevant here is that errors in contract logic and assumptions at the call boundary can have persistent consequences. This project operationalizes a narrow class of such mistakes—missing caller authorization and missing validation—rather than attempting to reproduce the survey's full attack taxonomy.

Luu et al. introduced Oyente, an analysis approach for detecting known smart-contract vulnerabilities using symbolic execution [6]. Tsankov et al. presented Securify, a static-analysis approach that reasons about compliance and violation patterns in contract behavior [7]. Static analysis can scale across source code and flag patterns, but tool output is detector- and model-dependent; absence of a finding does not prove a security property. The project's Slither run illustrates this distinction: it reported no finding for either identity contract although tests exposed unauthorized actions in IdentityVulnerable.

2.3 Comparative literature analysis

Table 1. Selected literature and standards relevant to the implemented study

Source	Method / contribution	Relevance to this project	Boundary or limitation
Mühle et al. [3]	Survey of essential SSI components.	Frames why a mapping-based prototype is only one part of an identity ecosystem.	Survey-level architecture; not the project's contract test design.

Source	Method / contribution	Relevance to this project	Boundary or limitation
W3C DID Core [2]	Normative DID syntax, data model, operations and resolution.	Defines what a DID system entails beyond an address-keyed mapping.	The project does not implement DID methods or resolution.
W3C VC Data Model 2.0 [4]	Normative issuer-holder-verifier credential model.	Clarifies distinction between credential documents and registry metadata.	The registry does not issue a standards-encoded VC.
Atzei et al. [5]	Survey/taxonomy of Ethereum contract attacks.	Motivates explicit threat assumptions and caller-bound checks.	Broad taxonomy; not a controlled paired evaluation of these contracts.
Luu et al. [6]	Oyente symbolic analysis for known contract vulnerabilities.	Illustrates automated analysis as a complement to testing.	Different analysis tool and vulnerability set.
Tsankov et al. [7]	Securify static analysis based on security patterns.	Supports discussion of detector coverage and false negatives.	Static results do not replace stateful scenario tests.

2.4 Synthesis and research gap

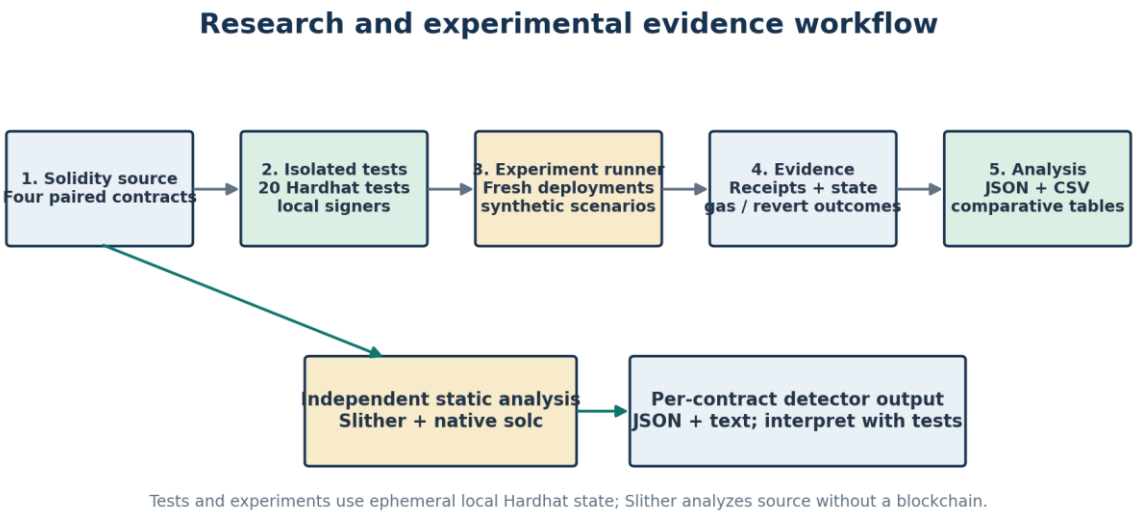
The literature establishes both the complexity of decentralized identity ecosystems and the value of automated contract analysis. The bounded gap addressed in this implementation is pedagogical and empirical rather than a claim that no prior work exists: the repository combines paired vulnerable/secure identity and credential contracts, targeted tests, receipt/state evidence, gas measurements, a local frontend and an independent static-analysis record. It shows, within a reproducible local setup, how targeted transaction tests can expose a security property missed by a particular static-analysis run.

3. METHODOLOGY

3.1 Research design

The study uses a controlled, paired comparative design. Each scenario invokes corresponding vulnerable and secure contracts using synthetic data and separate roles (owner, authorized issuer, attacker, subject). The dependent observations are transaction success or revert, receipt hash, gas used, and selected state before/after. Reverted calls are deliberately included in the experiment runner with a gas limit so the local chain can mine them and provide receipts; this is not equivalent to a production attack cost or a probability estimate.

Figure 1. Research and experimental evidence workflow



Source: Project scripts, Hardhat tests and the recorded Slither analysis.

3.2 Research questions and evaluation criteria

RQ1 and RQ2 are evaluated by whether an attacker transaction changes vulnerable state but reverts and preserves secure state. RQ3 is evaluated by paired acceptance/rejection and test assertions. RQ4 is evaluated using receipt gas units from the stored JSON/CSV run; comparisons are descriptive and single-run. RQ5 compares Slither detector output with the test suite's demonstrated behavior. For unauthorized operations, the stored report defines four unique classes (identity modification, identity deletion, credential issuance and credential revocation); "authorization bypass" is not double-counted as a separate operation.

3.3 Development environment and tools

Table 2. Verified implementation environment and responsibilities

Component	Observed configuration	Role in the implementation
Operating system / IDE	Windows; Visual Studio Code environment specified for the project.	Development and local command execution. Hardware model and memory were not recorded.

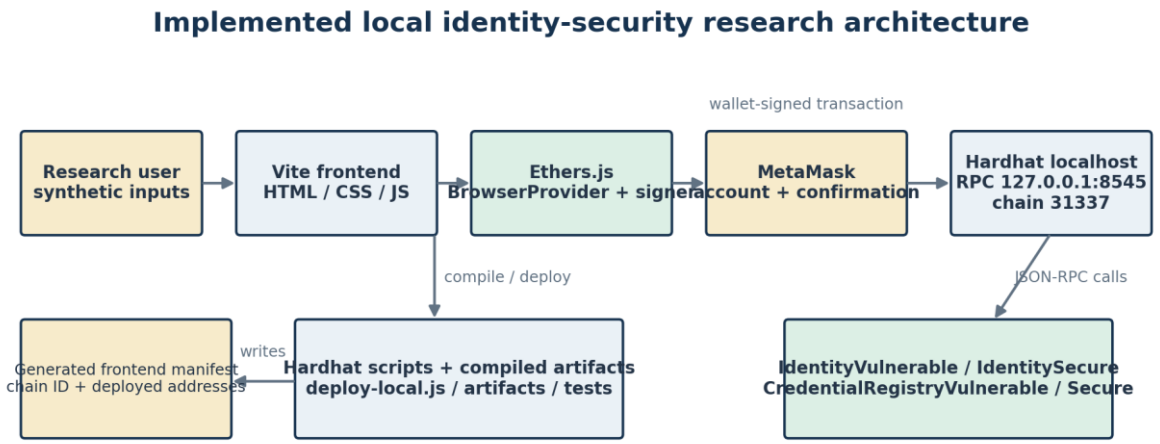
Component	Observed configuration	Role in the implementation
Solidity / compiler	pragma 0.8.24; Hardhat compiler 0.8.24; optimizer enabled, 200 runs.	Contract implementation and compilation.
Hardhat / ethers	Hardhat 2.26.3; hardhat-ethers 3.0.8; Ethers.js 6.13.4.	Local node, tests, deployments, experiment measurements and wallet provider.
Frontend	Vite 8.3.5; JavaScript modules; HTML/CSS; Ethers.js 6.13.4.	Local UI, wallet connection, reads, and signer-based transactions.
Local network	Hardhat localhost RPC http://127.0.0.1:8545; chain ID 31337.	Deterministic development/test environment; not a public Ethereum network.
Static analysis	Slither 0.11.6; native solc 0.8.24; 102 detectors in the recorded run.	Source-file-scoped static analysis; no blockchain transaction.
Runtime versions	README records Node.js 22.14.0; Slither README records Python 3.13.5.	Reproduction context; browser/MetaMask version was not recorded.

The Solidity security guide is relevant to the risk of treating public contract code as trusted merely because it compiles [8]. Hardhat's documented local network model and Ethers.js v6 BrowserProvider explain the repository's chain and injected-wallet integration [9], [10]. Slither is a source-level static-analysis tool; its findings here are interpreted as tool output rather than a proof of safety [11].

3.4 System architecture and workflow

The browser obtains an injected EIP-1193 wallet, uses Ethers.js BrowserProvider and a wallet signer, and submits reads or writes to configured contract addresses. The frontend deployment manifest is generated by the local deployment script and carries chain ID and contract addresses. Hardhat exposes the localhost JSON-RPC endpoint. Tests and experiment scripts use Hardhat signers and fresh contract instances, independently of MetaMask. The experiment runner writes JSON/CSV evidence; Slither analyzes Solidity source files separately.

Figure 2. Implemented frontend, wallet, local chain and contract architecture



Source: Inspection of frontend/src/main.js, frontend/deployments.local.json, blockchain/hardhat.config.js and blockchain/scripts.

3.5 Contract pairing and implementation method

The vulnerable contracts are intentionally retained as controlled negative examples. IdentityVulnerable writes identities[subject] in updateIdentity and deletes it in deleteIdentity after existence checks, but does not require the caller to equal the subject. Its registration overwrites the caller's record and accepts empty data. IdentitySecure requires subject == msg.sender for update and deletion, prevents duplicate registration, and validates a 1–64-byte label and nonzero commitment.

CredentialRegistryVulnerable has unrestricted issueCredential and revokeCredential functions and performs no issuance validation; repeated identifiers can be overwritten. CredentialRegistrySecure sets its deployer as administrator and initially authorizes that administrator as issuer. onlyIssuer gates issuance; administrator-only setIssuer rejects the zero address. Issuance validates nonzero identifier and subject, future expiry and uniqueness. Revocation is restricted to the administrator or recorded issuer, and verification checks existence, revocation and expiry.

3.6 Experimental procedures

- Compile the pinned Solidity sources with Hardhat and deploy fresh paired contract instances on the ephemeral Hardhat test chain.
- Create deterministic owner, issuer, attacker and subject roles; authorize the issuer on the secure credential registry.
- Perform successful baseline registration and owner-update operations and collect their gas receipts.
- Attempt unauthorized identity modification and deletion; capture transaction outcome and state before/after.
- Attempt empty identity registration, unauthorized credential issuance/revocation and malformed credential issuance; inspect resulting state.
- Test credential validity before and at the expiry boundary using local EVM timestamp controls.
- Write experiment records to reports/identity-comparison.json and reports/identity-comparison.csv; separately execute Slither per source file and preserve JSON/text summaries.

The automated suite creates fresh deployments in beforeEach hooks and asserts contract behavior and state. The local smoke test separately checks one localhost deployment's registration, read and authorized deletion using a Hardhat signer. MetaMask prompts require an injected browser wallet and are not covered by that smoke test.

3.7 Data collection, metrics and analysis

The primary evidence is the checked-in JSON and CSV report generated by the experiment runner. Each experiment stores expected outcomes, actual outcomes, success flags, receipt hashes, gas used, failure text and state snapshots. The runner also records the network chain ID, test counts and gas for baseline registration/owner update. The test-results JSON stores the test execution timestamp, exit code and console output. Slither's summary records each contract, JSON analysis success, process return code and detector metadata.

- Unauthorized vulnerable success rate = vulnerable unauthorized calls succeeding / four unique unauthorized operation classes.
- Secure unauthorized block rate = secure calls rejected / the same four operation classes. These are counts in one run, not population estimates.
- Gas comparison = receipt gas units for paired operations. No fiat conversion, gas-price adjustment, repeated-trial average or confidence interval is claimed.
- Static-analysis comparison = recorded findings by detector, impact and confidence; findings are interpreted with source code and test behavior.

3.8 Research ethics, safety and reproducibility

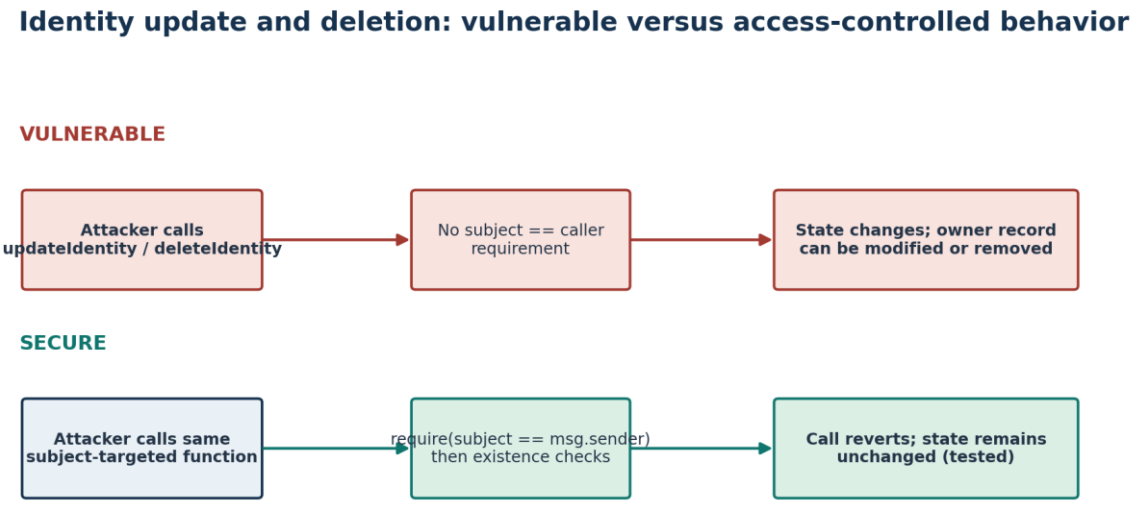
All transactions use local deterministic Hardhat accounts and synthetic labels/commitments. The deliberately vulnerable contracts are research artifacts and must not be deployed to public networks. No user identity data or private keys belong in the repository or report. Reproduction commands and pinned package versions are documented in the repository README and Slither README. The report does not reproduce local wallet private keys or treat local transaction hashes as public-chain evidence.

4. OBSERVATIONS

4.1 Identity contract behavior observed in source

Both identity contracts expose registerIdentity(string,bytes32), updateIdentity(address,string,bytes32), deleteIdentity(address), getIdentity(address) and verifyIdentity(address). The mapping is private at the Solidity interface level, but public contract state remains inspectable through storage/transaction analysis; “private” does not provide confidentiality. getIdentity deliberately exposes the display label, commitment and existence flag. IdentityRegistered, IdentityUpdated and IdentityDeleted events identify relevant subjects and actors according to each function’s event design.

Figure 3. Tested identity modification/deletion authorization boundary

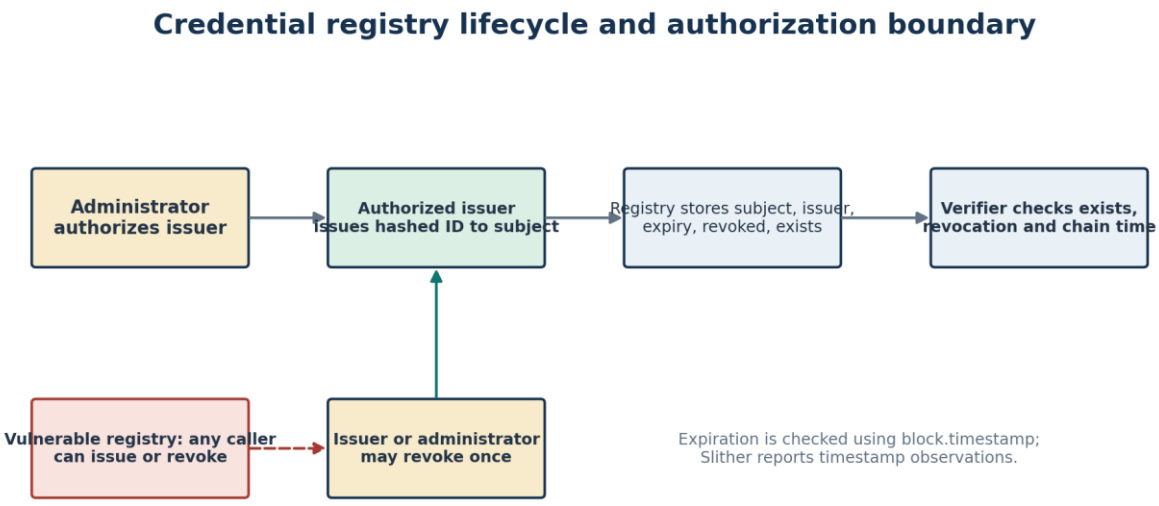


Source: Control flow in IdentityVulnerable.sol and IdentitySecure.sol; outcomes corroborated by tests and experiment receipts.

4.2 Credential registry behavior observed in source

The secure registry stores a credential keyed by bytes32 and returns subject, issuer, expiry, revoked and exists. Verification is true only when the record exists, is not revoked and current block time is before expiresAt. Administrator authority can authorize or deauthorize issuers; only the recorded issuer or administrator can revoke a record. The vulnerable registry stores the same basic structure but omits issuer gates and input validation, and any caller can revoke an existing credential.

Figure 4. Credential issuance, verification, revocation and expiry controls



Source: CredentialRegistryVulnerable.sol and CredentialRegistrySecure.sol; red dashed path denotes the tested vulnerable permission boundary.

4.3 Frontend and wallet integration observed

The Vite interface presents Identity, Credentials and Comparison Lab panels. Identity actions use the secure contract by default; the lab includes explicit vulnerable/secure experiment controls. The frontend loads deployments.local.json, validates chain ID 31337 and constructs Ethers.js contract instances using inline human-readable ABI fragments. Writes are attached to a signer obtained from MetaMask's injected provider; the activity area displays submission/confirmation or failure text. The current source requests account access, switches/adds Hardhat Local where needed, checks configured contract bytecode and distinguishes wallet/revert/insufficient-funds conditions. The user must still approve wallet prompts.

A source-level issue was that ethers' default BrowserProvider behavior could reject a network change from chain 1 to 31337. The project source was updated to use BrowserProvider "any", query the live injected chain ID and resynchronize before using the signer. Development-only registration diagnostics were added. This is implementation evidence, not proof of manual MetaMask execution: no verified browser-wallet acceptance screenshot or manual transaction record is present in the repository.

4.4 Deployment and chain observations

Hardhat configuration defines both the in-process hardhat network and localhost RPC at http://127.0.0.1:8545 with chain ID 31337. The deployment script refuses a different chain,

deploys all four contracts, waits for each receipt, verifies bytecode and writes addresses plus deployment transaction hashes to frontend/deployments.local.json. This manifest is ephemeral by design: after a node reset, deployment must be rerun and the manifest regenerated. The report records chain configuration and deployment workflow rather than assuming addresses remain valid indefinitely.

4.5 Automated test coverage observed

The checked-in test suite contains 20 tests: 11 identity tests and 9 credential tests. It includes positive flows (registration, owner update/delete, authorized issuance, issuer revocation and expiry) and negative security assertions (unauthorized caller, invalid fields, duplicate identifiers). The experiment report records the suite result as 20 passing and zero failing. Appendix B reproduces the test inventory; it does not imply tests for unimplemented standards features.

5. RESULTS

5.1 Recorded test and experiment result summary

The stored report “Identity Contract Security Comparison” was generated at 2026-10-25T05:59:31.794Z on Hardhat chain ID 31337. It records 20 passing and 0 failing automated tests, seven paired experiments and four unique unauthorized-operation classes. The four tested unauthorized operations succeeded on vulnerable contracts and were blocked on secure counterparts, yielding 4/4 (100%) and 4/4 (100%) for this finite local run. These figures are descriptive of the selected scenarios, not a statistical guarantee or universal vulnerability rate.

Table 3. Paired outcomes and receipt gas from the recorded experiment run

Scenario	Vulnerable outcome / gas	Secure outcome / gas	Tested condition
unauthorized-modification	SUCCEEDED / 37457	BLOCKED / 24890	Missing access control permits identity modification
unauthorized-deletion	SUCCEEDED / 31097	BLOCKED / 22080	Missing access control permits identity deletion
empty-input-validation	ACCEPTED / 50308	REJECTED / 24286	Missing validation accepts an empty name and zero commitment
unauthorized-credential-issuance	SUCCEEDED / 69090	BLOCKED / 24854	Missing issuer authorization permits forged credential issuance
credential-input-validation	ACCEPTED / 48518	REJECTED / 24193	Missing credential validation accepts zero identifiers, zero subjects, and expired credentials
unauthorized-credential-revocation	SUCCEEDED / 28673	BLOCKED / 24325	Missing revocation authorization permits denial of credential validity
credential-expiration	EXPIRED / 69090	EXPIRED / 71665	Credential validity must end at the recorded expiration timestamp

For the four unauthorized scenarios, vulnerable state changed as intended by the attacker, whereas secure state remained unchanged. Secure calls have receipt gas figures because the local configuration permits reverted experiment transactions to be mined; a revert therefore does not mean zero gas. Empty identity input was accepted by the vulnerable version and rejected by the secure version. Malformed credential input was likewise accepted versus rejected. Both registries made an expiring credential invalid at the tested boundary.

5.2 Access-control and input-validation observations

- Identity modification: vulnerable update succeeded (37,457 gas); secure update reverted (24,890 gas), and the subject’s original state remained unchanged.
- Identity deletion: vulnerable deletion succeeded (31,097 gas); secure deletion reverted (22,080 gas), preserving the secure record.
- Credential issuance: attacker issuance succeeded on the vulnerable registry (69,090 gas); secure issuance was blocked (24,854 gas).
- Credential revocation: attacker revocation succeeded on the vulnerable registry (28,673 gas); secure revocation was blocked (24,325 gas), leaving the valid credential active.
- Identity malformed input: empty label plus zero commitment was accepted on the vulnerable version (50,308 gas) and rejected on the secure version (24,286 gas).

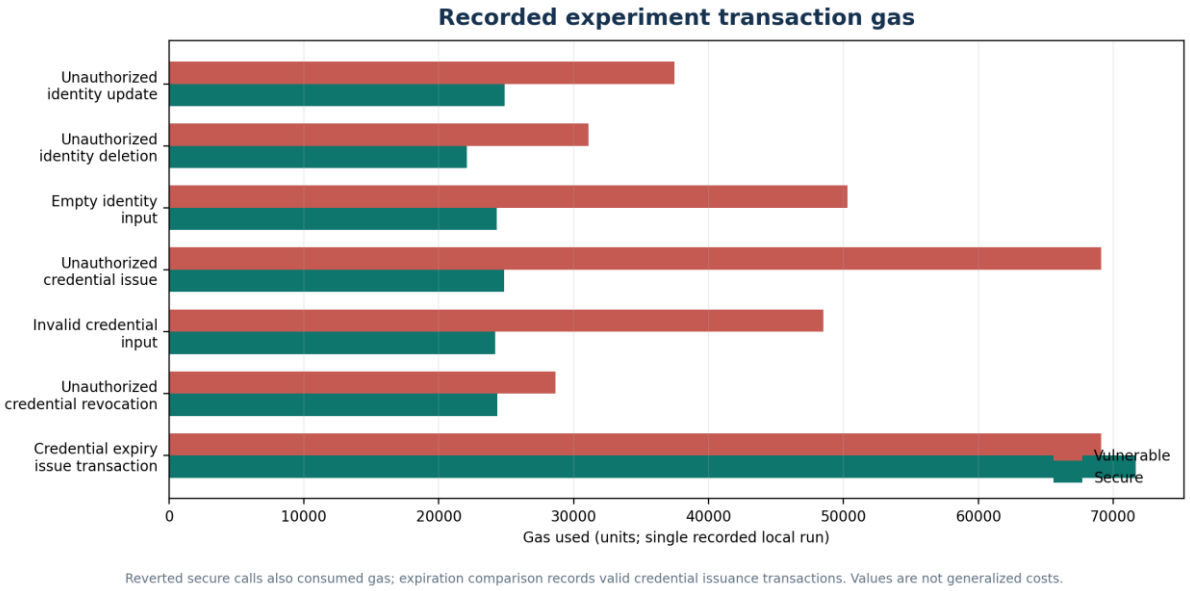
- Credential malformed input: zero identifier, zero subject and zero expiry were accepted on the vulnerable version (48,518 gas) and rejected on the secure version (24,193 gas).

5.3 Gas observations

Table 4. Recorded successful baseline gas use

Operation	Vulnerable gas	Secure gas	Difference	Relative difference
Identity registration	90856	91177	321	0.353%
Owner identity update	37553	37711	158	0.421%

Figure 5. Gas used by paired transactions in the recorded run



Source: reports/identity-comparison.json; one local execution per scenario.

Successful identity registration used 90856 gas on IdentityVulnerable and 91177 on IdentitySecure (difference 321 gas, approximately 0.353% relative to the vulnerable baseline). Successful owner update used 37553 versus 37711 gas (158 gas, approximately 0.421%). These small single-run deltas should not be generalized to all inputs or EVM versions. Failed secure calls consume measured gas as well; the chart does not equate gas with a currency fee.

5.4 Static-analysis results

Table 5. Recorded Slither findings by contract

Contract	Findings	JSON success	Finding summary
IdentityVulnerable	0	True	No findings in recorded output
IdentitySecure	0	True	No findings in recorded output
CredentialRegistryVulnerable	1	True	timestamp (Low, Medium)
CredentialRegistrySecure	2	True	timestamp (Low, Medium); timestamp (Low, Medium)

Slither reported zero findings for both identity contracts, one low-impact/medium-confidence timestamp finding in CredentialRegistryVulnerable and two low-impact/medium-confidence timestamp findings in CredentialRegistrySecure. Those credential findings

correspond to comparisons with block.timestamp in issuance/verification. They flag dependence on chain time; exact high-value timing decisions must account for bounded block-producer timestamp influence. The output did not identify the tested missing identity authorization. Thus, the experiment supports a detector-coverage limitation for this code/configuration, not an assertion that Slither is generally ineffective.

The recorded Slither summary marks all four JSON reports as analysisSucceeded=true while Windows return codes for the two credential reports appear as 4294967295 (unsigned representation of -1). The project Slither README explains that the wrapper treats the parsed Slither JSON success flag as authoritative and preserves both values. This platform-specific status anomaly is disclosed rather than silently interpreted as a clean zero exit code.

5.5 Figure and evidence interpretation

The charts and tables are derived from the committed report artifact generated at the timestamp shown above. Transaction hashes in Appendix A identify local Hardhat experiment receipts from that run; they are not mainnet transactions. The run is reproducible only by rerunning the local experiment workflow, which will generate new hashes and may produce different gas values as tools or configuration change. No screenshot was included in the available project evidence, so the report uses source-grounded architecture and workflow diagrams rather than fabricated UI captures.

6. CONCLUSION

6.1 Research summary

This implemented research project compares intentionally vulnerable and access-controlled identity and credential contracts on a local Hardhat network. Source code and tests demonstrate that omitted authorization permits non-owner identity modification/deletion and unauthorized credential issuance/revocation, while subject- and issuer-scoped checks block those test calls. Basic validation and uniqueness rules also differentiate the secure implementations. The stored report contains seven paired scenarios, 20 passing automated tests and transaction-level state/gas evidence.

6.2 Achievement of objectives and answers to research questions

RQ1: yes for the two tested identity operations; an arbitrary attacker altered/deleted vulnerable records but could not alter/delete the secure subject's record. RQ2: yes for tested issuance and revocation; vulnerable calls succeeded and secure calls were blocked. RQ3: yes for tested empty/zero identity data, malformed credential data and the duplicate/overlength unit tests; secure rules reject the specified cases, while vulnerable versions accept selected malformed/duplicate cases. RQ4: the stored single-run receipt values show small increases for successful secure registration/update and measurable gas for reverted calls; they do not establish generalized performance. RQ5: Slither's recorded output reports timestamp observations but misses the identity access-control defect demonstrated by tests.

6.3 Major findings and contribution

- Caller identity must be checked at the state-changing function boundary; accepting a target subject argument without binding it to msg.sender enabled the tested identity attacks.
- Issuer authorization and explicit revocation authority constrained the tested credential lifecycle actions.
- Input validation and uniqueness checks prevented the tested invalid/duplicate states but do not by themselves establish correctness or privacy.
- Static-analysis output and dynamic tests provide different evidence; neither replaces careful threat modeling and state-based assertions.
- The research contribution is a bounded, reproducible comparison with preserved vulnerable teaching variants, source, tests and transaction-level reports.

6.4 Limitations and future work

The prototype uses a local deterministic chain, synthetic data and a small set of attacks. It has no production key lifecycle, recovery, DID resolution, standards-compliant credential format, selective disclosure, privacy-preserving proof, independent auditor reproduction or statistical gas study. Browser-wallet flows are present in source, but manual MetaMask approval was not documented or independently verified for this academic evidence set. Slither findings are tied to the recorded tool/compiler version and static detectors. Future work can add repeated randomized test generation, invariant/property-based tests, formal verification, additional static analyzers with labeled ground truth, real DID/VC interoperability tests, privacy and data-minimization analysis, and separately documented manual browser-wallet tests on a disposable local account.

7. DISCUSSION

7.1 Interpretation in relation to prior work

The results are consistent with the general smart-contract security concern that public callable interfaces must enforce explicit authority and that tools analyze only the properties they model [5]–[7]. The contrast between state-changing tests and Slither findings demonstrates why passing a static analysis should not be reported as proof of absence of a vulnerability. The identity false negative is specific to the recorded Slither configuration and contracts; it does not benchmark detector quality across the field.

The identity literature and W3C standards place the contract within a larger ecosystem involving identifier resolution, controllers, issuers, holders and verifiers [2]–[4]. The project's simple mapping and registry are valuable for isolated authorization experiments but lack the protocol, key-management, privacy and interoperability properties of a full decentralized identity deployment. It would be inaccurate to label the implemented records as fully self-sovereign identities or verifiable credentials.

7.2 Security improvement and gas trade-off

Within this dataset, secure authorization changes observed unauthorized success to rejection and leaves the protected state intact. The secure checks add 321 gas to registration and 158 gas to owner update in the recorded baseline. That modest change is a descriptive local observation, not a universal security-overhead constant: storage state, calldata, compiler, optimizer and EVM revision influence gas. A production design would evaluate security and operational cost under realistic workloads, while recognizing that a reverted unauthorized operation still consumes gas paid by its sender.

7.3 Privacy, data integrity and lifecycle considerations

The project deliberately uses synthetic values, but its data model illustrates a broader privacy limitation. The display label is stored in contract state and returned by getIdentity; mapping visibility does not make it private. A deterministic hash of a predictable label may be guessed, and public addresses and event data can be correlated. Deletion removes the current mapping value but cannot erase historical transactions or prior state from the ledger. A production system should avoid personal attributes on-chain, minimize linkable metadata, use off-chain protected claims and carefully designed proofs, and review data-retention requirements. These concerns align with data-minimization and privacy cautions in credential standards [4].

7.4 Validity, reproducibility and generalization

The paired local design improves internal comparability: each attacker operation has an equivalent secure contract action, synthetic state and explicit assertion. However, the measurements are not repeated across independent clients, chains, compiler versions or hardware; no confidence intervals or non-parametric tests are appropriate. The report records artifact timestamps and hashes to establish provenance. External validity is intentionally limited to the four implemented contracts and selected test conditions. The local Hardhat chain does not reproduce production consensus, network latency or account-management behavior.

7.5 Evidence gaps and submission readiness

The repository provides automated contract tests, experiment reports, deployment scripts, Slither outputs and frontend source. It does not contain a verified browser screenshot or a manual MetaMask transaction transcript. The frontend's wallet interaction should therefore be described as implemented in source, while on-chain test and smoke-test results remain separate evidence. Student identity, registration number, guide details and institutional signatures are absent and remain placeholders. The supplied template also uses two department labels; official wording must be confirmed before submission.

8. REFERENCES

[1] D. Yaga, P. Mell, N. Roby, and K. Scarfone, "Blockchain Technology Overview," NIST Interagency/Internal Report 8202, National Institute of Standards and Technology, Gaithersburg, MD, USA, Oct. 2018, doi: 10.6028/NIST.IR.8202.

[2] W3C Decentralized Identifier Working Group, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 19, 2022. [Online]. Available: https://www.w3.org/TR/2022/REC-did-core-20220719/.

[3] A. Mühle, A. Grüner, T. Gayvoronskaya, and C. Meinel, "A survey on essential components of a self-sovereign identity," Computer Science Review, vol. 30, pp. 80–86, Nov. 2018, doi: 10.1016/j.cosrev.2018.10.002.

[4] W3C Verifiable Credentials Working Group, "Verifiable Credentials Data Model v2.0," W3C Recommendation, May 15, 2025. [Online]. Available: https://www.w3.org/TR/vc-data-model-2.0/.

[5] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," in Principles of Security and Trust, Lecture Notes in Computer Science, vol. 10204. Berlin, Germany: Springer, 2017, pp. 164–186, doi: 10.1007/978-3-662-54455-6_8.

[6] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making Smart Contracts Smarter," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2016, pp. 254–269, doi: 10.1145/2976749.2978309.

[7] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2018, pp. 67–82, doi: 10.1145/3243734.3243780.

[8] Solidity Team, "Security Considerations," Solidity Documentation, v0.8.24. [Online]. Available: https://docs.soliditylang.org/en/v0.8.24/security-considerations.html.

[9] Nomic Foundation, "Hardhat documentation: Networks and local development network," Hardhat 2 documentation. [Online]. Available: https://hardhat.org/docs.

[10] ethers.js Contributors, "Ethers.js v6 documentation: BrowserProvider and contract interaction." [Online]. Available: https://docs.ethers.org/v6/.

[11] Trail of Bits, "Slither: Static Analyzer for Solidity," documentation and source repository. [Online]. Available: https://github.com/crytic/slither.

APPENDICES

Appendix A. Transaction receipt evidence from the stored local experiment report

The following hashes and gas readings are copied from reports/identity-comparison.json, generated 2026-10-25T05:59:31.794Z on the local Hardhat network (chain ID 31337). These are local-chain receipts from a disposable experimental run. They are not mainnet or persistent public-chain evidence.

Table 6. Local experiment transaction hashes and measured gas

Scenario	Variant	Transaction hash	Gas used	Outcome
unauthorized-modification	Vulnerable	0xf70fea894e9340d625fc87ab360c511ad82440600e3f10959014d527f4585458	37457	SUCCEEDED
unauthorized-modification	Secure	0x575ad0d4e93054a2f74aa75ff1d7b5d9ab66e6de9d13b44e8f7847e7854e55e6	24890	BLOCKED
unauthorized-deletion	Vulnerable	0xe03963c546eba1986502fcfa5f9153b4948874cbcb492e5902c19e9bb81a7d50	31097	SUCCEEDED
unauthorized-deletion	Secure	0x3fb636ccf02eab2eda54ef123aa8265a307986101226a233ed146a4a3d939f52	22080	BLOCKED
empty-input-validation	Vulnerable	0x8969f207ff1d8bdad9e05d97772478f1b82b705e7a65814f28f2e716263f863d	50308	ACCEPTED
empty-input-validation	Secure	0x7e7f8d4e941fed93f2991433b2790e946dfca158fac4996df4a0dc670c847840	24286	REJECTED
unauthorized-credential-issuance	Vulnerable	0x366d29dd87db4de75e083d9717262cf28f5ea3fa0862a37523400db6376b8837	69090	SUCCEEDED
unauthorized-credential-issuance	Secure	0xd851b02db6c117a635292532a8d6543dfb46948ebf435f5208a63ec0d74deeb0	24854	BLOCKED
credential-input-validation	Vulnerable	0x7f69100033fc992a98705adae188aa75588d69c0cb4e840558e26eb19d9af76b	48518	ACCEPTED
credential-input-validation	Secure	0x8de262ea6b95c118e0936a8b33661619216d098993e456e17bbdebf6a349c37c	24193	REJECTED
unauthorized-credential-revocation	Vulnerable	0xe491eecc325831ebd01596b26a49edb919b212e885a685a0cab1df4e76be36cb	28673	SUCCEEDED
unauthorized-credential-revocation	Secure	0xcfb4f04a7ca526b39518dee59fca2e75583bb40372fdcec4dce156d918d9e174	24325	BLOCKED
credential-expiration	Vulnerable	0x8372b043adbc7a4276dcd542241b29a28ce7bcbaf2c5db9152c79ebcd3968447	69090	EXPIRED
credential-expiration	Secure	0x2dd2f4078aed94d1bfc438669324b95483d68fd6772029c083c36812f00f2157	71665	EXPIRED

Appendix B. Automated test inventory

Table 7. Test cases present in the checked-in Hardhat suite

ID	Suite	Test case
I01	Identity	registers and verifies an identity on both contracts
I02	Identity	allows an unauthorized identity modification in the vulnerable contract
I03	Identity	blocks an unauthorized identity modification in the secure contract
I04	Identity	allows an unauthorized identity deletion in the vulnerable contract
I05	Identity	blocks an unauthorized identity deletion in the secure contract
I06	Identity	allows the identity owner to update and delete a secure identity
I07	Identity	accepts empty identity data in the vulnerable contract
I08	Identity	rejects empty names and zero commitments in the secure contract
I09	Identity	rejects overlong identity labels in the secure contract
I10	Identity	rejects duplicate registration in the secure contract
I11	Identity	allows duplicate registration in the vulnerable contract
C01	Credential	allows an authorized issuer to issue a subject-bound credential
C02	Credential	permits credential forgery on the vulnerable registry and blocks unauthorized issuance
C03	Credential	allows unauthorized revocation on the vulnerable registry and blocks it securely
C04	Credential	rejects invalid hashes, subjects, and already-expired credentials
C05	Credential	accepts malformed credential data in the vulnerable registry
C06	Credential	rejects duplicate credential identifiers
C07	Credential	automatically reports credentials as invalid after expiry in both registries
C08	Credential	restricts issuer authorization changes to the registry administrator
C09	Credential	limits credential revocation to its issuer or the administrator

Appendix C. Representative source excerpts

The excerpts below are illustrative and abbreviated from implemented source. Full source files remain in blockchain/contracts/.

```
IdentityVulnerable.updateIdentity: require(identities[subject].exists, "Identity does not exist"); // no subject/caller equality check

IdentitySecure.updateIdentity: require(subject == msg.sender, "Not identity owner"); require(identities[subject].exists, "Identity does not exist");

CredentialRegistrySecure.issueCredential: external onlyIssuer; requires nonzero credentialId/subject, future expiresAt, and an unused credentialId.

CredentialRegistrySecure.revokeCredential: require(msg.sender == administrator || msg.sender == credential.issuer, "Not credential issuer");
```

Appendix D. Reproduction and submission-completion notes

Local reproducibility commands from PowerShell (use separate terminals for the node and browser):

```
Set-Location C:\developer\identity-project\blockchain
npm run compile
npm test
npm run research

# Terminal 1: start local RPC
Set-Location C:\developer\identity-project\blockchain
npm run node

# Terminal 2: deploy/update manifest and smoke-test
Set-Location C:\developer\identity-project\blockchain
npm run deploy:local
npm run smoke:local

# Terminal 3: start Vite frontend
Set-Location C:\developer\identity-project\frontend
npm run dev
```

Submission checklist (details were not present in the provided materials and are not guessed):

- Replace [STUDENT NAME] and [TO BE PROVIDED] registration-number placeholders.
- Confirm guide/supervisor name, designation, department, and required university certificate wording; obtain real signatures and approval.
- Personalize acknowledgements only for assistance actually received; review and sign the declaration only if each assertion is true.
- Resolve the template's department-name inconsistency: title-page wording uses Department of Forensic Science / School of Sciences, while a certificate header uses Department of Cyber Security & Digital Forensics.
- Confirm whether the university requires page numbering, roman numerals for preliminary pages, a different academic year/date, or PDF export. The supplied template has blank headers/footers and no page-number fields. Refresh contents/list page-reference fields in Microsoft Word before printing.

- Provide hardware specifications if required. CPU, RAM, browser version, MetaMask version, manual MetaMask evidence and project screenshots were not present in the inspected repository.
- Manually verify MetaMask on a disposable funded Hardhat account if required, and capture genuine screenshots/transaction records only after performing that verification.